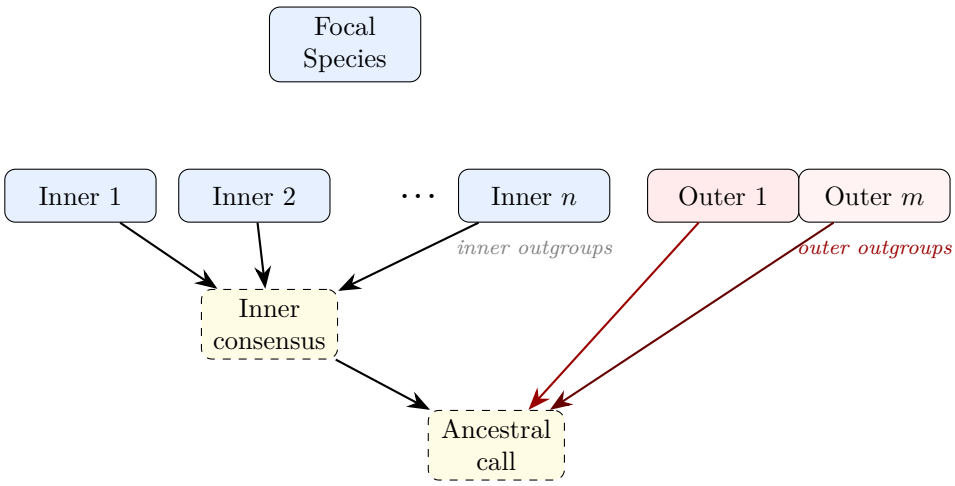
ancify

Ancestral Allele Polarization Pipeline for Any Species

A species-agnostic, config-driven Python package
for inferring ancestral alleles using outgroup alignments



Version 1.0 — Generated: February 19, 2026

Contents

1 Introduction

1.1 Why Polarize Variants?

In population genetics, most analyses of the **site frequency spectrum** (SFS) require knowing whether an allele is *ancestral* (inherited from the common ancestor) or *derived* (arose by mutation). Without polarization, the SFS is *folded*—one cannot distinguish a rare derived allele from a rare ancestral allele—which limits the power of tests for selection, demographic inference, and mutation-rate estimation.

Polarization is required for:

- **Unfolded SFS**: used by ∂a∂i, moments, `mushi`, and similar demographic inference tools.
- **Tests of selection**: McDonald–Kreitman, Fay & Wu's H, D_n/D_s, etc.
- **Ancestral recombination graphs**: correct orientation of mutations on genealogies.
- **Mutation spectra**: directional mutation-rate estimation requires knowing which allele is ancestral.

1.2 The Outgroup Approach

The standard approach compares the focal species to one or more **outgroup** species. If an outgroup carries allele X and the focal species is polymorphic for X and Y, parsimony suggests X is ancestral. Using *multiple* outgroups reduces misinference from back-mutations, convergent evolution, and incomplete lineage sorting (ILS).

1.3 What This Package Does

The `ancify` package generalises this logic into a species-agnostic, config-driven pipeline:

1. **Phase 1 — Projection**: converts pairwise net-AXT alignments from any number of outgroup species into FASTA files in focal-species coordinates.
2. **Phase 2 — Ancestral calling**: infers the ancestral allele at every position using a two-tier inner/outer outgroup voting scheme with case-encoded confidence.
3. **Phase 3 — Evaluation** (optional): compares the calls against a reference ancestral sequence and/or VCF variant data.

The pipeline works for **any focal species** for which pairwise net-AXT alignments to outgroup species are available (typically from the UCSC Genome Browser).

2 Biological Background

2.1 Inner vs. Outer Outgroups

The pipeline partitions outgroup species into two tiers:

Inner outgroup One or more *closely related* species. A majority vote among them produces the **inner consensus**. Because these species diverged recently from the focal species, most of their genome is identical to the true ancestral state.

Outer outgroup One or more *distantly related* species. They serve as an independent check: if the inner consensus agrees with the outer outgroup, the call is made with high confidence.

Example: Human Polarization

For *Homo sapiens* (hg38):
- Inner: bonobo (∼6 Mya), chimpanzee (∼6 Mya), gorilla (∼9 Mya)
- Outer: rhesus macaque (∼25 Mya)

If all three great apes agree on allele `A` and macaque also carries `A`, the ancestral call is `A` with high confidence.

Example: Drosophila Polarization

For *D. melanogaster* (dm6):
- Inner: *D. simulans*, *D. sechellia*
- Outer: *D. yakuba*

Same algorithm, different species tree.

2.2 Net Alignments and the AXT Format

The pipeline uses **net alignments** from the UCSC Genome Browser—one-to-one best-chain pairwise alignments stored in **AXT format**. Each alignment block consists of four lines:

1. Header: index, target chromosome, start, end, query chromosome, start, end, strand, score.
2. Target (focal) aligned sequence (with gap characters `-`).
3. Query (outgroup) aligned sequence.
4. Blank separator.

These files can be downloaded from UCSC for hundreds of species pairs.

3 Installation

3.1 From Source

```
cd ancify/
pip install .
```

This installs the `ancify` command-line tool and the `ancify` Python package.

3.2 With Evaluation Dependencies

The evaluation phase (Phase 3) requires `scikit-allel` and `matplotlib`:

```
pip install '.[evaluate]'
```

3.3 Core Dependencies

Table 1: Package dependencies.

Package	Purpose	Required?
`pyyaml`	Configuration file parsing	Yes
`numpy`	Array operations in evaluation	Yes
`scikit-allel`	VCF reading (Phase 3)	Optional
`matplotlib`	Plotting (Phase 3)	Optional

4 Package Architecture

4.1 Module Layout

```
ancify/                          # project root
|-- pyproject.toml               # build metadata & dependencies
|-- example_configs/
|   |-- hg38_bcgm.yaml          # human worked example
|   |-- mouse_example.yaml      # mouse hypothetical example
|   |-- drosophila_example.yaml
|-- ancify/                      # importable Python package
    |-- __init__.py
    |-- __main__.py              # python -m ancify
    |-- cli.py                   # command-line interface
    |-- config.py                # YAML config loading & validation
    |-- utils.py                 # FASTA I/O, majority_vote, helpers
    |-- project.py               # Phase 1: coordinate projection
    |-- ancestral.py             # Phase 2: ancestral state inference
    |-- evaluate.py              # Phase 3: evaluation & QC
```

Listing 1: Package file structure.

4.2 Three-Phase Pipeline

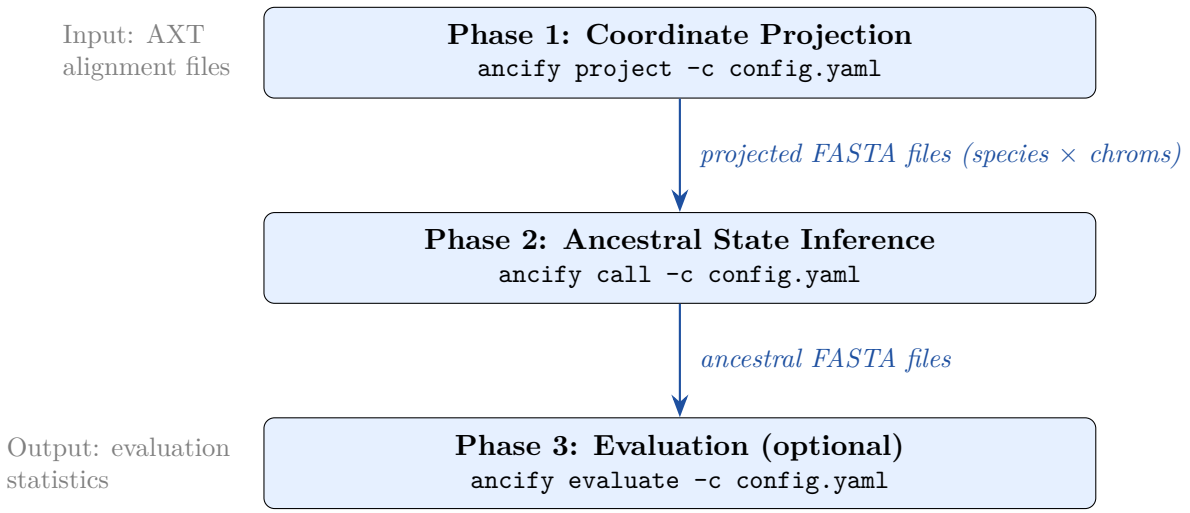


Figure 1: Three-phase pipeline, each invocable independently or together via `ancify run`.

5 Configuration

All pipeline behaviour is controlled by a single **YAML configuration file**. Generate a template with:

```
ancify init -o config.yaml
```

5.1 Configuration Reference

```yaml
# Informal label for the focal species.
focal_species: human

# Tab-separated file: chrom_name <TAB> length [<TAB> ...]
chromosome_lengths: chromoLens.txt

# Optional: restrict to specific chromosomes.
# If omitted, all entries in the lengths file are used.
chromosomes:
  - chr1
  - chr2
  - chrX

outgroups:
  # Inner: closely related species (majority vote).
  inner:
    - name: bonobo
      alignment: hg38.panPan3.net.axt.gz
    - name: chimp
      alignment: hg38.panTro6.net.axt.gz
    - name: gorilla
      alignment: hg38.gorGor6.net.axt.gz
  # Outer: distantly related species (independent check).
  outer:
    - name: macaque
      alignment: hg38.rheMac10.net.axt.gz

# Intermediate files go here (projected/ subdirectory).
work_dir: .

# Final ancestral FASTA files go here.
output_dir: ./ancestral_calls

# Minimum allele count for the majority-vote consensus.
min_inner_freq: 1
min_outer_freq: 1

# Parallel workers (one chromosome per worker).
num_cpus: 24

# Optional evaluation block.
evaluation:
  reference_dir: ./reference_ancestral/
  # {chrom} = full name, {chrom_id} = without "chr" prefix
  reference_pattern: "ancestor_{chrom_id}.fa"
  vcf_dir: ./vcf/
  vcf_pattern: "variants.{chrom}.vcf.gz"
```

Listing 2: Annotated configuration file.

5.2 Key Configuration Fields

Table 2: Configuration fields and their meaning.

Field	Required	Description
`focal_species`	Yes	Label for the focal species (cosmetic).
`chromosome_lengths`	Yes	Path to tab-separated lengths file.
`chromosomes`	No	Subset of chromosomes to process; defaults to all.
`outgroups.inner`	Yes	List of inner outgroup species (name + alignment path).
`outgroups.outer`	Yes	List of outer outgroup species (name + alignment path).
`work_dir`	No	Working directory for projected sequences (default: `.`).
`output_dir`	No	Output directory for ancestral FASTAs (default: `./ancestral_calls`).
`min_inner_freq`	No	Minimum count for inner majority vote (default: 1).
`min_outer_freq`	No	Minimum count for outer majority vote (default: 1).
`num_cpus`	No	Parallel workers (default: 4).
`evaluation`	No	Optional block for Phase 3 settings.

5.3 Pattern Placeholders

Evaluation filename patterns support two placeholders:

- `{chrom}` — the full chromosome name as it appears in the config (e.g. `chr1`, `2L`).
- `{chrom_id}` — the name with any leading `chr` prefix stripped (e.g. `1`, `X`).

6 Phase 1: Coordinate Projection

6.1 Purpose

Transforms each outgroup's pairwise alignment (AXT format) into a FASTA file where every position corresponds to a position in the focal-species reference. Unaligned positions are filled with `N`.

6.2 Algorithm (`ancify.project`)

1. Load focal-species chromosome lengths from the config.
2. For each outgroup × chromosome combination:
   (a) Allocate a byte array of length L (chromosome length), pre-filled with `N`.
   (b) Stream through the AXT file. For each alignment block matching the target chromosome, walk the aligned human sequence position by position:

- If the focal character is a nucleotide (`A/T/C/G/N`), store the corresponding outgroup character at the focal coordinate.
- If the focal character is a gap (`-`), skip (insertion in the outgroup, not in focal coordinates).

(c) Write the resulting sequence as a FASTA file.

```
seq = bytearray(b"N" * chrom_length)
with open(axt_path, "rt") as f:
    for block in parse_axt_blocks(f):
        if block.target_chrom == target_chrom:
            pos = block.start
            for h, q in zip(block.target_seq, block.query_seq):
                if h.upper() in "ATCGN":
                    seq[pos - 1] = ord(q)
                    pos += 1
return seq.decode("ascii")
```

Listing 3: Core projection logic (simplified from `project.py`).

6.3 Output

Phase 1 creates `<work_dir>/projected/<species>/<chrom>.fa` for every outgroup species and chromosome.

Parallelization uses Python's `ProcessPoolExecutor` with one worker per task (species × chromosome).

7 Phase 2: Ancestral State Inference

7.1 The Generalized Algorithm

For each genomic position, the algorithm performs two majority votes and then compares:

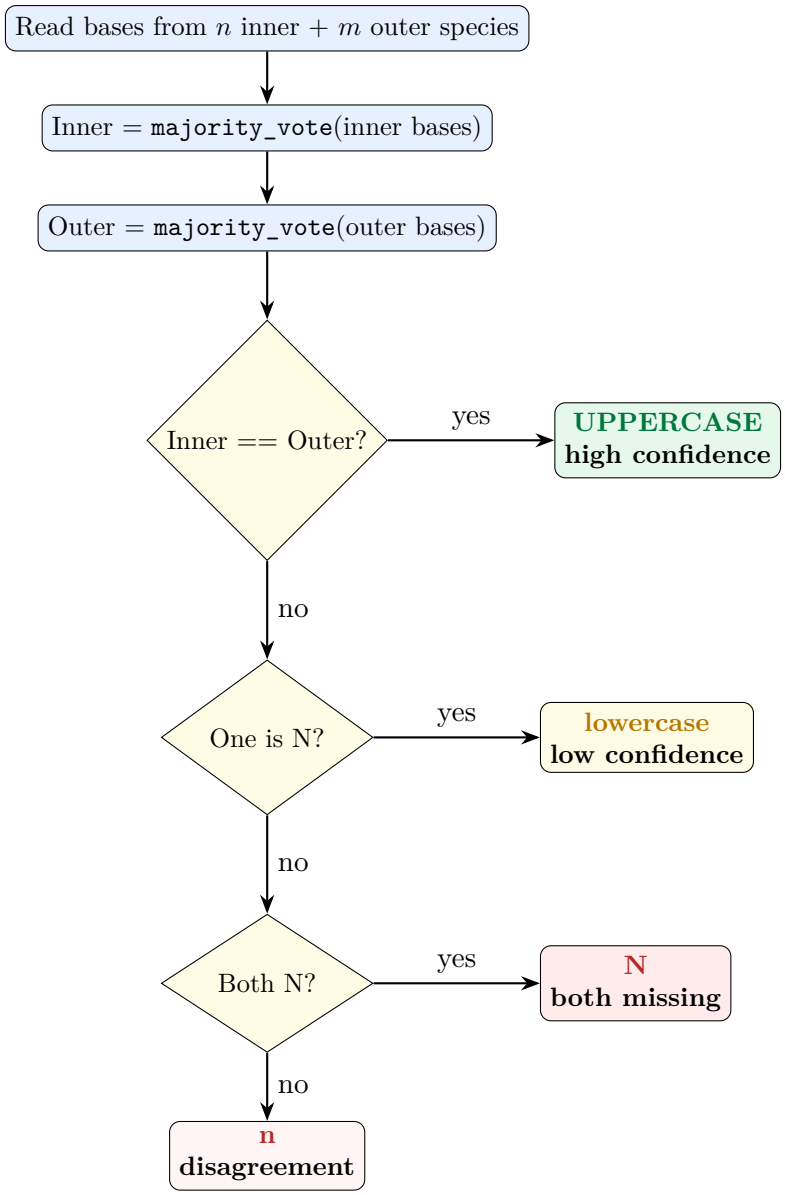


Figure 2: Decision flowchart for the generalized ancestral calling algorithm.

7.2 Core Function

```python
def call_ancestral_base(inner_bases, outer_bases,
                        min_inner_freq=1, min_outer_freq=1):
    inner = majority_vote(inner_bases, min_inner_freq)
    outer = majority_vote(outer_bases, min_outer_freq)

    if inner != "N" and inner == outer:
        return inner                # HIGH confidence (uppercase)
    if inner == "N" and outer != "N":
        return outer.lower()        # LOW confidence (lowercase)
    if inner != "N" and outer == "N":
        return inner.lower()        # LOW confidence (lowercase)
    if inner == "N" and outer == "N":
        return "N"                  # Both missing
    return "n"                      # Disagreement
```

Listing 4: `call_ancestral_base()` from `ancestral.py`.

7.3 The `majority_vote()` Function

```python
VALID_ALLELES = ("A", "C", "G", "T")

def majority_vote(bases, min_freq=1):
    upper = [b.upper() for b in bases]
    counts = {a: upper.count(a) for a in VALID_ALLELES}
    best = max(VALID_ALLELES, key=lambda a: counts[a])
    if counts[best] < min_freq:
        return "N"
    return best
```

Listing 5: `majority_vote()` from `utils.py`.

Ties are broken in alphabetical order (A, C, G, T) for reproducibility. The function works for any number of input species ($n \geq 1$).

7.4 Output Encoding

Table 3: Confidence encoding in the output FASTA.

Character	Confidence	Condition
`A`, `C`, `G`, `T`	High	Inner and outer outgroups agree
`a`, `c`, `g`, `t`	Low	Only one outgroup tier has data
`n` (lowercase)	Unresolved	Inner and outer disagree
`N` (uppercase)	Missing	Both tiers lack data

Why Letter Case?

Using uppercase for high-confidence and lowercase for low-confidence calls is a compact encoding within standard FASTA format. Downstream tools can filter by confidence level: for stringent analyses, restrict to uppercase bases (inner–outer agreement); for maximum coverage, include lowercase as well. This convention is compatible with the Ensembl EPO ancestral sequences.

8 Phase 3: Evaluation

8.1 Available Metrics

The evaluation module computes three categories of metrics (the latter two are optional):

Coverage statistics (always): proportion of high-confidence, low-confidence, and missing positions.

Reference comparison (optional): agreement rate between the pipeline's calls and a reference ancestral sequence (e.g. Ensembl EPO).

VCF comparison (optional): concordance of ancestral calls with REF/ALT alleles at known variant sites.

8.2 Output

For each chromosome, a summary file is written to `<output_dir>/evaluation/<chrom>.evaluation.txt` with sections:

```
[coverage]
  total_positions: 248956422
  high_confidence: 197145832
  low_confidence: 25203651
  missing: 26606939
  prop_nonmissing: 0.8931
  prop_high_confidence: 0.7918

[reference_comparison]
  test_nonmissing: 0.7912
  ref_nonmissing: 0.9016
  both_nonmissing: 5234112
  agreement_rate: 0.9992
  disagreement_rate: 0.0008

[vcf_comparison]
  num_variants: 6102435
  prop_nonmissing: 0.8965
  matches_ref: 0.9453
  matches_alt: 0.0494
  matches_either: 0.9948
```

Listing 6: Example evaluation output.

9 Command-Line Interface

9.1 Available Commands

Table 4: CLI commands.

Command	Description
`ancify init [-o FILE]`	Generate a template configuration file.
`ancify project -c CONFIG`	Run Phase 1 (coordinate projection).
`ancify call -c CONFIG`	Run Phase 2 (ancestral calling).
`ancify evaluate -c CONFIG`	Run Phase 3 (evaluation).
`ancify run -c CONFIG`	Run all phases end-to-end.

9.2 Global Options

```
ancify -v run -c config.yaml      # verbose (debug) logging
ancify run -c config.yaml -n 8    # override num_cpus to 8
```

The package can also be invoked as a module:

```
python -m ancify run -c config.yaml
```

10 Worked Example: Human (hg38) BCGM

This section walks through the original use case that motivated the package: polarizing the human genome using bonobo, chimpanzee, gorilla, and macaque.

10.1 Phylogenetic Context

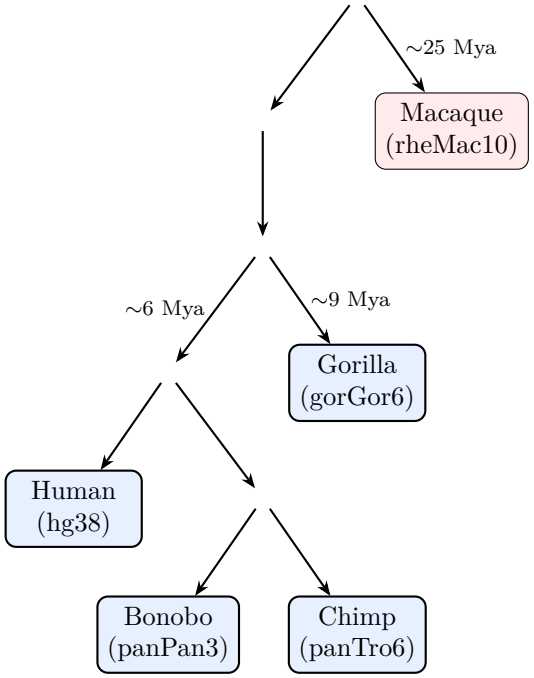


Figure 3: Great ape phylogeny for the human BCGM pipeline.

10.2 Configuration

The example config is provided in `example_configs/hg38_bcgm.yaml`:

```yaml
focal_species: human
chromosome_lengths: chromoLens.txt

outgroups:
  inner:
    - name: bonobo
      alignment: hg38.panPan3.net.axt.gz
    - name: chimp
      alignment: hg38.panTro6.net.axt.gz
    - name: gorilla
      alignment: hg38.gorGor6.net.axt.gz
  outer:
    - name: macaque
      alignment: hg38.rheMac10.net.axt.gz

output_dir: ./human_ancestor_bcgm
num_cpus: 24

evaluation:
  reference_dir: ./embi_homo_sapiens_ancestor_GRCh38
  reference_pattern: "homo_sapiens_ancestor_{chrom_id}.fa"
  vcf_dir: /path/to/1000genomes/
```

```
  vcf_pattern: "ALL.chr{chrom_id}...vcf.gz"
```

Listing 7: Key sections of the hg38 BCGM config.

10.3 Input Data

Table 5: Input alignment files for the human BCGM pipeline.

File	Species	Size
`hg38.panTro6.net.axt.gz`	Chimpanzee	1.67 GB
`hg38.panPan3.net.axt.gz`	Bonobo	1.66 GB
`hg38.gorGor6.net.axt.gz`	Gorilla	1.66 GB
`hg38.rheMac10.net.axt.gz`	Macaque	1.40 GB

These are downloaded from the UCSC Genome Browser:

```
wget https://hgdownload.soe.ucsc.edu/goldenPath/hg38/\
     vsPanTro6/hg38.panTro6.net.axt.gz
```

10.4 Running the Pipeline

```
ancify run -c example_configs/hg38_bcgm.yaml
```

Or phase by phase:

```
ancify project  -c hg38_bcgm.yaml    # 2-8 hours
ancify call     -c hg38_bcgm.yaml    # 30-60 minutes
ancify evaluate -c hg38_bcgm.yaml    # 20-40 minutes
```

10.5 Evaluation Results

The BCGM calls have been evaluated against the Ensembl EPO (13-primate) ancestral reference:

Table 6: BCGM vs. Ensembl EPO for selected human chromosomes (high-confidence only).

Metric	chr1	chr22	chrX
Non-missing (BCGM)	79.1%	74.6%	44.4%
Non-missing (EMBI)	90.2%	81.2%	44.1%
Disagreement rate	0.083%	0.109%	3.51%
BCGM matches REF/ALT	99.6%	99.5%	99.3%
EMBI matches REF/ALT	99.5%	99.5%	100.0%

Key Observations

- BCGM and EMBI agree at >99.9% of autosomal sites where both call.
- EMBI has higher coverage because it uses a 13-species multiple alignment.
- Including low-confidence calls raises BCGM coverage from ∼79% to ∼90% on chr1

(with disagreement increasing from 0.08% to 0.57%).
- Both methods achieve >99.3% concordance with 1000G REF/ALT alleles.

11 Adapting to Other Species

11.1 What You Need

To polarize a new focal species you need:

1. **A reference genome assembly** for the focal species.
2. **A chromosome-lengths file**: a tab-separated file with at least two columns (chromosome name, length in bp).
3. **Pairwise net-AXT alignment files** for each outgroup species. These are widely available from the UCSC Genome Browser's download site for many vertebrate and insect assemblies.
4. **Phylogenetic knowledge** to decide which species are "inner" (closely related) and which are "outer" (distantly related).

11.2 Choosing Inner vs. Outer Outgroups

Design Guidelines

- **Inner outgroups** should diverge from the focal species *more recently* than the outer outgroup. They provide the primary ancestral signal.
- **Outer outgroups** should be *far enough* that convergent substitutions with the inner outgroup are rare. A divergence of >2× the inner divergence is a useful rule of thumb.
- Using ≥2 inner outgroups enables majority voting, which is robust to single-species alignment gaps or lineage-specific substitutions.
- Multiple outer outgroups can also be used; they undergo their own majority vote.

11.3 Example: Mouse

```
focal_species: mouse
chromosome_lengths: mm39.chromLens.txt

outgroups:
  inner:
    - name: rat
      alignment: mm39.rn7.net.axt.gz
  outer:
    - name: rabbit
      alignment: mm39.oryCun2.net.axt.gz

output_dir: ./mouse_ancestral
num_cpus: 8
```

Listing 8: Hypothetical mouse configuration.

11.4 Example: *Drosophila melanogaster*

```yaml
focal_species: drosophila_melanogaster
chromosome_lengths: dm6.chromLens.txt

chromosomes: [2L, 2R, 3L, 3R, 4, X]

outgroups:
  inner:
    - name: simulans
      alignment: dm6.droSim2.net.axt.gz
    - name: sechellia
      alignment: dm6.droSec1.net.axt.gz
  outer:
    - name: yakuba
      alignment: dm6.droYak3.net.axt.gz

output_dir: ./dmel_ancestral
num_cpus: 6
```

Listing 9: Hypothetical *Drosophila* configuration.

Note that the chromosome names (`2L`, `3R`, etc.) do not have a `chr` prefix—the package handles any naming convention.

11.5 Creating a Chromosome-Lengths File

For any UCSC assembly, chromosome sizes can be obtained with:

```
mysql --user=genome --host=genome-mysql.soe.ucsc.edu -A \
  -e "SELECT chrom, size FROM chromInfo" dm6 > dm6.chromLens.txt
```

Or from a FASTA index:

```
samtools faidx reference.fa
cut -f1,2 reference.fa.fai > chromLens.txt
```

12 Using the Output Downstream

12.1 Looking Up an Ancestral Allele

```python
from ancify.utils import read_fasta

_, seq = read_fasta("ancestral_calls/chr1.fa")

position = 1000000  # 1-based
allele = seq[position - 1]

print(f"Ancestral: {allele}")
print(f"High confidence: {allele.isupper() and allele in 'ACGT'}")
```

Listing 10: Python: look up ancestral allele at a genomic position.

12.2 Polarizing a VCF File

```
for variant in vcf:
    anc = ancestral_seq[variant.POS - 1]

    if anc.upper() in "ACGT":
        if anc.upper() == variant.REF:
            variant.INFO["AA"] = variant.REF    # REF is ancestral
        elif anc.upper() == variant.ALT:
            variant.INFO["AA"] = variant.ALT    # ALT is ancestral
        else:
            variant.INFO["AA"] = "."            # no match
    else:
        variant.INFO["AA"] = "."                # missing

    confidence = "high" if anc.isupper() else "low"
```

Listing 11: Pseudocode for VCF polarization.

12.3 Filtering by Confidence

```
# High confidence only
if anc.isupper() and anc in "ACGT":
    use_variant()

# Including low confidence
if anc.upper() in "ACGT":
    use_variant()

# Skip unresolved disagreements
if anc.lower() == "n":
    skip_variant()
```

Listing 12: Confidence-level filtering.

12.4 Programmatic API

The package can be used as a library, not just via the CLI:

```
from ancify.config import load_config
from ancify.project import run_projection
from ancify.ancestral import run_ancestral_calling
from ancify.ancestral import call_ancestral_base

# Run the full pipeline
cfg = load_config("config.yaml")
run_projection(cfg)
run_ancestral_calling(cfg)

# Or call the core function directly
base = call_ancestral_base(
    inner_bases=["A", "A", "G"],    # 3 inner species
    outer_bases=["A"],              # 1 outer species
)
# Returns "A" (high confidence: inner majority A == outer A)
```

Listing 13: Using the package programmatically.

13 Theoretical Considerations

13.1 Misinference from Incomplete Lineage Sorting

When ancestral population sizes were large relative to the time between speciation events, gene trees can disagree with the species tree (**ILS**). In such regions, the majority vote among inner outgroups may reflect the derived state. The outer outgroup serves as a safeguard: disagreement between tiers is flagged as `n` (unresolved).

ILS is more common in species with:

- Large ancestral N_e (effective population size).
- Short inter-node intervals on the species tree.
- Rapid radiations.

For the human–chimp–gorilla clade, ILS affects ∼1–3% of the genome.

13.2 Back Mutations and Convergence

A substitution along the outer outgroup lineage creates a false disagreement. Convergent mutations in both the outer and focal lineages create false agreement on the wrong ancestral state. Both events are rare ($\ll$1% of sites for typical mammalian substitution rates) but contribute to the ∼0.1% disagreement rate observed in practice.

13.3 Alignment Quality

The pipeline's accuracy depends fundamentally on alignment quality. Net alignments mitigate paralog confusion by selecting one-to-one best chains, but regions with segmental duplications, transposable elements, or structural variants remain challenging and typically appear as `N` (missing) in the projected sequences.

13.4 The `min_inner_freq` / `min_outer_freq` Parameters

Increasing these thresholds enforces stricter consensus requirements:

Table 7: Effect of `min_inner_freq` with 3 inner outgroups.

Value	Meaning
1	Any single inner species suffices (maximum coverage, lower stringency).
2	At least 2 of 3 must agree (balances coverage and accuracy).
3	All 3 must agree (maximum stringency, reduced coverage).

14 Troubleshooting

14.1 Memory Issues

`MemoryError` during Phase 2 Each chromosome loads $n + m$ full-length FASTA sequences into memory simultaneously. For human chr1 (∼249 Mb) with 4 outgroups, this is roughly 1.2 GB per chromosome. When running chromosomes in parallel, memory usage scales as num_cpus × $(n + m) \times L$. Reduce `num_cpus` to limit concurrent workers, or process chromosomes sequentially with `num_cpus: 1`.

`MemoryError` during Phase 1 (Projection) AXT files for closely related species (e.g. chimpanzee) can contain millions of alignment blocks. The projection algorithm builds a position-indexed byte array per chromosome, which is memory-efficient, but if many chromosomes are processed in parallel the aggregate footprint may exceed available RAM. Reduce `num_cpus` or project one species at a time.

Slow Phase 1 with large AXT files Each AXT file (∼1.5–1.7 GB compressed for great apes) must be streamed sequentially per chromosome. Expect 2–8 hours for a full primate set on a single machine. If wall time is a concern, split projection jobs by species (each AXT file is independent) and run them on separate nodes.

14.2 Input Data Issues

AXT parsing errors Verify file integrity with `gzip -t file.axt.gz`. Ensure files are in standard UCSC net-AXT format (4-line blocks: header, target sequence, query sequence, blank line). Files from non-UCSC sources may use different conventions or include extra header lines; inspect the first few records manually.

`ValueError`: unexpected target base This occurs when the target (focal) sequence in an AXT block contains a character other than A, T, C, G, N, or `-`. Some older or non-standard AXT files include IUPAC ambiguity codes (e.g. R, Y, M). Pre-filter these by converting ambiguity codes to N before running the pipeline, or patch the AXT file.

Sequence length mismatch in Phase 2 All projected FASTA files for the same chromosome must have identical length (equal to the value in the chromosome-lengths file). This error typically means:

- AXT files were generated against *different genome builds* (e.g. one is hg38, another is hg19). All alignments must be to the same focal assembly.
- The chromosome-lengths file does not match the assembly used for the AXT alignments. Regenerate it with `samtools faidx` or the UCSC `chromInfo` table.
- A projected FASTA file was truncated due to a previous interrupted run. Delete the partial output and re-run Phase 1.

Chromosome not found in AXT file If a chromosome listed in the config is absent from an AXT file, the projection produces an all-N sequence for that species. This is expected for chrY (which may be absent or poorly assembled in some outgroup species) and for unplaced scaffolds. If it happens for major autosomes, verify the chromosome naming convention: UCSC uses `chr1`, `chrX`, etc., while Ensembl uses `1`, `X`.

Chromosome naming mismatches The chromosome names in the config, the chromosome-lengths file, and the AXT target fields must all be consistent. Common pitfalls:

- Mixing `chr1` (UCSC) with `1` (Ensembl).
- Trailing whitespace or invisible characters in the lengths file.
- Using `chrMT` vs. `chrM` for the mitochondrial genome.

Verify with: `head -5 chromoLens.txt | cat -A` (shows `$` at line ends and `^I` for tabs).

Corrupted or incomplete AXT downloads Large alignment files (>1 GB) sometimes get truncated during download. After downloading, always verify:

```
gzip -t hg38.panTro6.net.axt.gz && echo "OK"
```

If the file is corrupt, re-download from the UCSC Genome Browser.

14.3 Configuration Issues

YAML syntax errors `ancify` validates the config on startup. Common YAML mistakes include:

- Missing colons after keys.
- Inconsistent indentation (YAML requires spaces, not tabs).
- Unquoted strings that YAML interprets as other types (e.g. `chromosomes: [1, 2, X]` parses `1` and `2` as integers; use `["1", "2", "X"]` or `[chr1, chr2, chrX]`).

Validate syntax independently with: `python -c "import yaml; yaml.safe_load(open('config.yaml')`

Relative vs. absolute paths All paths in the config are resolved relative to the *current working directory* at runtime, not relative to the config file. Use absolute paths to avoid confusion, or always `cd` to the project directory before running the pipeline.

`min_inner_freq` too high Setting `min_inner_freq` = the number of inner outgroups requires *all* inner species to agree. This maximizes accuracy but dramatically reduces coverage (sites where any inner outgroup has a gap or N become missing). Start with the default value of 1 and increase after inspecting the coverage statistics from Phase 3.

14.4 Evaluation Issues

Missing evaluation data Phase 3 is optional. If reference or VCF data are unavailable, simply omit the `evaluation` block in the config. The pipeline will skip evaluation gracefully.

`ImportError: scikit-allel` VCF-based evaluation requires `scikit-allel`: install with `pip install 'ancify[evaluate]'`. On some systems, `scikit-allel` also requires `h5py` and `zarr`; install those separately if the build fails.

Evaluation reference pattern does not match any files Check that the `reference_pattern` uses the correct placeholder. `{chrom}` expands to the full name (e.g. `chr1`), while `{chrom_id}` strips the `chr` prefix (e.g. `1`). List the reference files to confirm the naming convention:

```
ls reference_ancestral/
# e.g. homo_sapiens_ancestor_1.fa -> use {chrom_id}
# e.g. ancestor_chr1.fa           -> use {chrom}
```

VCF files are not indexed `scikit-allel` can read unindexed VCF files but may be extremely slow for genome-wide data. Create a tabix index for each VCF:

```
tabix -p vcf variants.chr1.vcf.gz
```

High disagreement rate between pipeline and reference A disagreement rate >1% on autosomes likely indicates a data issue rather than a biological difference. Common causes:

- Different genome builds between the pipeline and the reference ancestral sequence.
- The reference ancestral sequence uses a different set of outgroup species or a different inference method (e.g. EPO multi-species alignment vs. pairwise parsimony).
- One-based vs. zero-based coordinate confusion when comparing at specific positions.

Compare at known invariant sites first (e.g. highly conserved exonic regions) to diagnose the issue.

14.5 Output and Downstream Issues

Very high proportion of N (missing) calls If >30% of an autosome is missing, check:

- That projection (Phase 1) completed successfully for all outgroups. Inspect projected FASTAs: `grep -c "N" projected/chimp/chr1.fa` should not return a value close to the chromosome length.
- Alignment coverage for the outgroup species. Distantly related or poorly assembled outgroups may cover only a fraction of the focal genome.
- Whether the chromosome is acrocentric (e.g. human chr13–15, 21–22) with large heterochromatic regions that lack outgroup alignments.

Lowercase calls where uppercase expected Lowercase indicates that only the inner *or* outer outgroup had data, not both. This is normal at the edges of alignment coverage and in repetitive regions. If the proportion of lowercase calls is unexpectedly high, one outgroup tier may have poor alignment coverage. Run `ancify evaluate` and inspect the per-tier coverage breakdown.

Unexpected `n` (disagreement) calls in conserved regions Disagreement between inner and outer outgroups in conserved exonic regions may indicate:

- Incomplete lineage sorting (biological; see Section 13).
- Alignment artifacts (e.g. paralogous alignments in duplicated genes).
- Assembly errors in one of the outgroup genomes.

Cross-check with the UCSC Genome Browser's conservation tracks or Ensembl's EPO alignment.

FASTA index (`.fai`) files Some downstream tools (e.g. `samtools`, `bcftools`) require an `.fai` index. Generate one with:

```
samtools faidx ancestral_calls/chr1.fa
```

Note that if the FASTA is written as a single long line (no line wrapping), some older tools may choke. In that case, reformat with `fold`:

```
fold -w 80 ancestral_calls/chr1.fa > chr1_wrapped.fa
samtools faidx chr1_wrapped.fa
```

Using output with `tskit` / tree sequences When adding ancestral states to a tree sequence, ensure the ancestral allele string matches the reference allele encoding. The pipeline outputs one character per reference position (0-indexed in Python, 1-indexed in genomic coordinates). For a variant at 1-based position p:

```
ancestral_allele = seq[p - 1]  # 0-based indexing
```

14.6 Platform-Specific Issues

macOS: `ulimit` errors with many parallel workers macOS has a low default file-descriptor limit. If you see "Too many open files" errors, increase the limit before running:

```
ulimit -n 4096
```

Cluster / HPC environments On SLURM or SGE clusters, request sufficient memory per task. A conservative estimate for Phase 2 is:

$$\text{Memory (GB)} \approx \texttt{num\_cpus} \times (n_{\text{inner}} + n_{\text{outer}} + 1) \times 0.3$$

where 0.3 GB accounts for the largest human chromosome (∼249 Mb) loaded as a Python string. Request 2× this estimate to leave headroom for overhead.

Python version compatibility The package requires Python ≥3.9 for `dict` ordering guarantees and type-hint syntax. If you encounter `SyntaxError` on older Python versions, upgrade or use a conda environment:

```
conda create -n ancify python=3.11
conda activate ancify
pip install .
```

15 Complete Module Reference

Table 8: Module and function reference.

Module / Function	Purpose
`ancify.config`	
`load_config(path)`	Load and validate a YAML config file.
`PipelineConfig`	Dataclass holding all pipeline settings.
`ancify.utils`	
`read_fasta(path)`	Read a single-record FASTA file.
`write_fasta(path, hdr, seq)`	Write a single-record FASTA file.
`read_chromosome_lengths(p)`	Parse a tab-separated lengths file.
`majority_vote(bases, min)`	Return majority nucleotide or `N`.
`chrom_id(chrom)`	Strip leading `chr` prefix.
`ancify.project`	
`project_alignment(...)`	Project one chromosome from AXT to FASTA.
`run_projection(config)`	Run Phase 1 for all species/chroms.
`ancify.ancestral`	
`call_ancestral_base(...)`	Infer ancestral allele at one position.

Module / Function	Purpose
	Run Phase 2 for all chromosomes.
`run_ancestral_calling(cfg)`	
`ancify.evaluate`	
	Coverage and confidence breakdown.
`compute_coverage_stats(seq)`	
	Agreement with reference ancestral.
`compare_to_reference(...)`	
`compare_to_vcf(...)`	Concordance with VCF REF/ALT.
	Run Phase 3 for all chromosomes.
`run_evaluation(config)`	
`ancify.cli`	
`main()`	Entry point for the `ancify` command.

A Quick Reference Card

Pipeline Quick Reference

Install:

```
pip install .                    # core
pip install '.[evaluate]'        # with evaluation deps
```

Generate template config:

```
ancify init -o config.yaml
```

Run all phases:

```
ancify run -c config.yaml
```

Run individual phases:

```
ancify project  -c config.yaml
ancify call     -c config.yaml
ancify evaluate -c config.yaml
```

Output encoding:
`ACGT` = high confidence | `acgt` = low confidence | `n` = disagreement | `N` = missing

B Comparison: General Package vs. Original Scripts

Table 9: What changed from the original hg38-specific scripts to the general package.

Aspect	**Original Scripts**	**`ancify` Package**
Species support	Human only (hg38)	Any focal species
Outgroup count	3 inner + 1 outer (fixed)	n inner + m outer (configurable)
Configuration	Hardcoded paths	YAML config file
Execution	Separate scripts + GNU Parallel	Single CLI with subcommands
Parallelization	Ray / GNU Parallel	`ProcessPoolExecutor` (stdlib)
Installation	Copy scripts	`pip install .`
Chromosome naming	`chr1`–`chrY` hardcoded	Any naming convention
Evaluation	Human-specific (EMBI + 1000G)	Optional, configurable patterns
Memory (Phase 1)	Dict per position	Bytearray (much more efficient)